

Arrays

Definition

1. An array is a collection of similar data elements have the same data type.
2. The elements of array are stored in consecutive memory locations and are referenced by an index (also known as subscript).
3. Subscript indicates an ordinal number of the elements counted from the beginning of the array.

Examples:

- List of temperatures recorded on everyday of the month.
- List of employee in a company.
- List of students in a class.
- List of products sold.
- List of customers.

Declaration of Arrays

An array must be declared before being used. Declaring an array means specifying three things:

Data Type: what kind of value it can store, for example, int, float , char , double.

Name: to identify the array.

Size: the maximum number of values that the array can hold.

Arrays are declared using the following syntax:

Type name[size];

For example : `int marks[10];` declares a marks variable to be an array containing 10 elements.

C++ does not allow declaring an array whose number of elements is not known at the compile time.

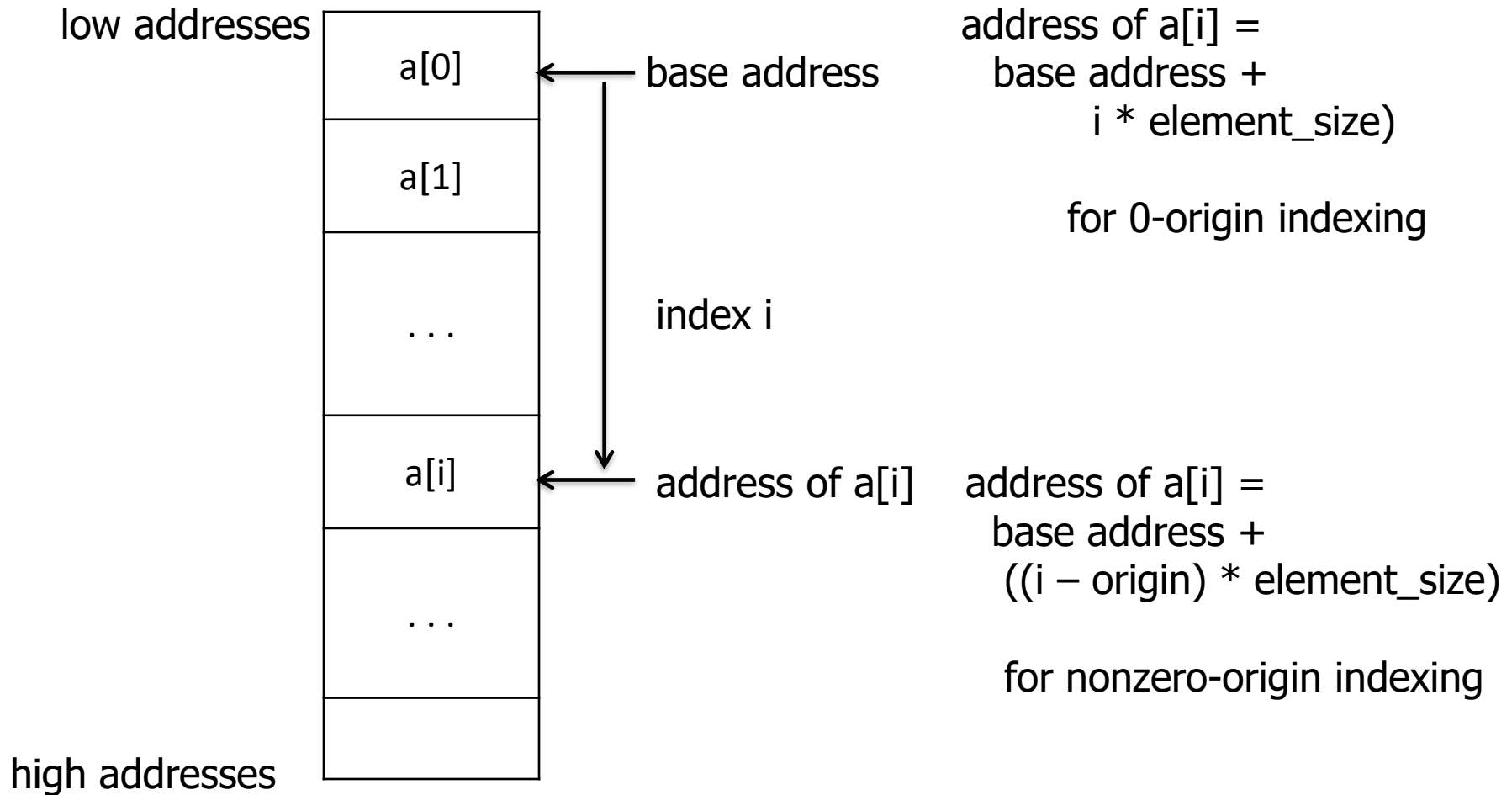
Accessing Elements of the Array

1. For accessing the individual element of the array, the array subscript must be used. e.g. to access the fourth element of the array, we must write `arr[3]`.
2. Subscript index must be an integral value or an expression that evaluates to integral value.
3. To access all the elements of the array, we must use a loop.

Calculating the Address of Array Elements

1. Array name is symbolic reference for the address to the first byte of the array. When we use the array name, we are actually referring to the first byte of array.
2. An array stores all its elements in consecutive memory locations . So, storing just the base address, i.e. the address of the first element in the array is sufficient.
3. The address of other data elements can simply be calculated using the base address.

Storage allocation



Storing Values in Arrays

When we declare and define an array , we are just allocating space for the elements, no values are stored in array. To store values in the array, there are three ways:

1. Initialize the elements
2. Input values for the elements
3. Assign values to the elements

Calculating the Length of the Array

Length of the array is given by the number of elements stored in it. The general formula to calculate the length of the array is,

$$\text{Length} = \text{upper bound} - \text{lower bound} + 1$$

Where upper bound is the index of the last element

And lower bound is the index of the first element in the array.

Operations that can be performed on Arrays

Following operations can be performed on arrays:

1. Traversal
2. Insertion
3. Search
4. Deletion
5. Merging

Traversal

Traversing an array element means accessing each and every element of the array for a specific purpose.

```
#include<iostream>
using namespace std;
int main()
{
    int a[20];
    int i, n;
    int large = -1;
    cout << "Enter the number of elements in the array:" << endl;
    cin >> n;
    cout << " Enter elements of array " << endl;
    for (i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    for (i = 0; i < n; i++)
    {
        if (a[i] >= large)
        {
            large = a[i];
        }
    }
    cout << "The largest number of array = " << large << endl;
    system("pause");
    return 0;
}
```

Insertion

1. Inserting an element to the array means adding a new data element in an already existing array.
2. If the element has to be inserted at the end of the existing array, then the task of insertion is quite simple.
3. Just add 1 to the upper_bound and assign the value. Here we assume that the memory space allocated for the array is still available.

For example:

If an array is declared to contain 10 elements , but currently is having only 8 elements, then obviously there is space to accommodate two more elements. However, if it already has 10 elements, then we will not be able to add another element to it.

- If we have to insert in the middle of an array , then this is not a trivial task . On an average, we might have to move as much as half of the elements from its position in order to accommodate space for the new element.

```
#include<iostream>
using namespace std;
int main()
{
    int a[10];
    int i, n, pos, num;
    cout << "Enter the number of elements in the array:" << endl;
    cin >> n;
    cout << " Enter elements of array " << endl;
    for (i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    for (i = 0; i < n; i++)
    {
        cout << " a[" <<i <<"] = "<<a[i] << endl;
    }
    cout << "Enter the number to be inserted : " << endl;
    cin >> num;
    cout << "Enter the position at which you want to insert an element in the array:" << endl;
    cin >> pos;
```

```
    for (i = n; i >= pos; i--)
    {
        a[i + 1] = a[i];
    }
    a[pos] = num;
    n++;
    cout << "Array Elements after insertion " << endl;
    for (i = 0; i < n; i++)
    {
        cout << a[i] << endl;
    }
    system("pause");
    return 0;}
```

Deletion

1. Deleting an element from the array means removing a data element from already existing array.
2. If the element has to be deleted from the end of the existing array, then deletion is quite simple. We just have to subtract 1 from the upper bound.
3. Consider an array , whose elements are arranged in ascending order .If an element have to be deleted from somewhere middle in the array.
4. First find the location from where the element has to be deleted and then move all the elements one space towards the left so that space vacated by the deleted element be occupied by rest of the elements.

```

#include<iostream>
using namespace std;
int main()
{
    int a[10];
    int i, n, pos, num;
    cout << "Enter the number of elements in the array:" << endl;
    cin >> n;
    cout << " Enter elements of array " << endl;
    for (i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    for (i = 0; i < n; i++)
    {
        cout << " a[" <<i <<"] = "<<a[i] << endl;;
    }
    cout << "Enter the position at which you want to delete an element in the array:" << endl;
    cin >> pos;
    for (i = pos; i <= n; i++)
    {
        a[i ] = a[i+1];
    }
    n--;
    cout << "Array Elements after insertion " << endl;
    for (i = 0; i < n; i++)
    {
        cout << a[i] << endl;
    }
    system("pause");
    return 0;}

```

Merging

1. First copy the contents of the first array into the third array and then copying the contents of second array into third array.
2. The merged array contains the contents of the first array followed by the contents of the second array.
3. If the arrays are unsorted then merging arrays is very simple as one just needs to copy the contents of one array into another.
4. Merging is not a trivial task when the two arrays are sorted and the merged array also needs to be sorted.

Merging two Unsorted Array

```
#include<iostream>
using namespace std;
int main()
{
    int a1[10],a2[10],a3[20];
    int i, n1, n2,m,index=0;
    cout << "Enter the number of elements in the array1 :" << endl;
    cin >> n1;
    cout << " Enter elements of first array " << endl;
    for (i = 0; i < n1; i++)
    {
        cin >> a1[i];
    }
    for (i = 0; i < n1; i++)
    {
        cout << " a1[" <<i <<"] = " <<a1[i] << endl;;
    }
    cout << "Enter the number of elements in the array2 :" << endl;
    cin >> n2;
    cout << " Enter elements of second array " << endl;
```

```
for (i = 0; i < n2; i++)
{
    cin >> a2[i];
}
for (i = 0; i < n2; i++)
{
    cout << " a2[" << i << "] = " << a2[i] << endl;
}
m = n1 + n2;
for (i = 0; i < n1; i++)
{
    a3[index] = a1[i];
    index++;
}
for (i = 0; i < n2; i++)
{
    a3[index] = a2[i];
    index++;
}
cout << "The merged array is :" << endl;
for (i = 0; i < m; i++)
{
    cout << a3[i] << endl;
}
system("pause");
return 0;}
```

Merging two Sorted Arrays

```
#include<iostream>
using namespace std;
int main()
{
    int a1[10],a2[10],a3[20];
    int i, n1, n2,m,index=0;
    int index_first = 0, index_second = 0;
    cout << "Enter the number of elements in the array1 :" << endl;
    cin >> n1;
    cout << " Enter elements of first array " << endl;
    for (i = 0; i < n1; i++)
    {
        cin >> a1[i];
    }
    for (i = 0; i < n1; i++)
    {
        cout << " a1[" <<i <<"] = " <<a1[i] << endl;;
    }
    cout << "Enter the number of elements in the array2 :" << endl;
    cin >> n2;
    cout << " Enter elements of second array " << endl;
    for (i = 0; i < n2; i++)
    {
        cin >> a2[i];
    }
}
```

```
for (i = 0; i < n2; i++)
{
    cout << " a2[" << i << "] = " << a2[i] << endl;
}
m = n1 + n2;
while(index_first < n1 && index_second < n2)
{
    if (a1[index_first] < a2[index_second])
    {
        a3[index] = a1[index_first];
        index_first++;

    }
    else
    {
        a3[index] = a2[index_second];
        index_second++;
    }
    index++;
}
```

```
if (index_first = n1)
{
    a3[index] = a2[index_second];
    index_second++;
    index++;
}
else if (index_second = n2)
{
    a3[index] = a1[index_first];
    index_first++;
    index++;
}
cout << "The merged array is :" << endl;
for (i = 0; i < m; i++)
{
    cout << a3[i] << endl;
}
system("pause");
return 0;}
```

Searching Array Elements

Searching means whether a particular value is present in the array or not.

If the value is present in the array then searching is said to be successful and the searching process gives the location of that value in the array.

If the value is not present in the array, the searching process displays the appropriate message and in this case searching is said to be unsuccessful.

Linear Search

```
#include<iostream>
using namespace std;
≡ int main()
{
    int a[10], num, i, n, found = 0, pos = -1;
    cout << "Enter the number of elements in the array : ";
        cin >> n;
    cout << "Enter the elements:" << endl;
    for (i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    for (i = 0; i < n; i++)
    {
        cout<< a[i]<<endl;
    }
    cout << "Enter the number that has to be searched:" << endl;
    cin >> num;
```

```
for (i = 0; i < n; i++)
{
    if (a[i] == num)
    {
        found = 1;
        pos = i;
        cout << "Number found at position"<< pos << endl;
        break;
    }
}

if (found == 0)
{
    cout << "Number does not exist:" << endl;
}

system("pause");
return 0;
```


Passing Arrays to Functions

Specify name without brackets

- To pass array **myArray** to **myFunction**

int myArray[24];

myFunction(myArray, 24);

- Array size usually passed, but not required
 - Useful to iterate over all elements

- Arrays passed-by-reference

- Functions can modify original array data
- Value of name of array is address of first element
 - Function knows where the array is stored
 - Can change original memory locations

- Individual array elements passed-by-value

- Like regular variables
- **square(myArray[3]);**

```
#include<iostream>
using namespace std;
void read_array(int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    for (int i = 0; i < n; i++)
    {
        cout<<a[i]<<endl;
    }
}
int main()
{
    int a[20];
    int n;
    cout << "enter the number of elements in the array:" << endl;
    cin >> n;
    read_array(a, n);
    system("pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
void display(int marks[5]);
int main()
{
    int marks[5] = { 88, 76, 90, 61, 69 };
    for (int i = 0; i < 5; i++)
    {
        cout << marks[i] << endl;
    }
    display(marks);
    for (int i = 0; i < 5; i++)
    {
        cout << marks[i] << endl;
    }
    system("pause");
    return 0;
}
```

```
void display(int m[5])
{
    cout << "Enter the marks:";
    for (int i = 0; i < 5; i++)
    {
        cin >> m[i];
    }

    cout << "Displaying marks: " << endl;

    for (int i = 0; i < 5; ++i)
    {
        cout << "Student " << i + 1 << ": " << m[i] << endl;
    }
}
```